Manual de ejecución

Asignación de Horarios Universitarios con GNU Prolog

1. Instalación

El programa funciona en Linux con GNU Prolog 1.4.5. En este proyecto se utiliza Linux desde WSL.

En Ubuntu o Debian se instalan GNU Prolog y las herramientas de compilación con el siguiente comando.

```
sudo apt install gprolog build-essential time
```

La versión se comprueba con `gprolog --version`. Los ejemplos siguientes se ejecutan desde la carpeta del proyecto.

2. Uso del programa

Para resolver una instancia se indican el archivo de entrada, el archivo de salida y la estrategia.

```
./horarios.sh instances/valid/small.dat salida.txt bt
```

La estrategia `bt` utiliza backtracking y `clpfd` utiliza dominios finitos. Sin opciones adicionales, el programa entrega el primer horario válido.

2.1. Opciones disponibles

Opción	Uso
`--heuristica=VALOR`	Cambia el orden de búsqueda. El valor inicial es `original`.
`--optimizar`	Busca el horario de menor costo.
`--limite=N`	Limita la búsqueda a `N` nodos. Sin esta opción no hay límite de nodos.
`--horarios-por-aula`	Incluye el horario de cada aula en el resultado.
`--exportar-prolog=RUTA`	Guarda también el horario en un archivo Prolog.

Se elige una sola heurística por ejecución.

Valor	Criterio
`original`	Mantiene el orden de lectura de las sesiones.
`mrv`	Elige la sesión con menos ubicaciones disponibles.
`degree`	Elige la sesión con más restricciones con otras sesiones.
`demand`	Prioriza los grupos con más inscritos y luego las sesiones más largas.
`lcv`	Prueba primero las ubicaciones que dejan más opciones a las otras sesiones.

El valor `ff` también está disponible con `clpfd` y equivale a `mrv` en este programa.

2.2. Ejemplos de ejecución

Backtracking con MRV y dominios finitos con `ff` se ejecutan de la siguiente manera.

```
./horarios.sh instances/valid/constraints.dat \
    salida-mrv.txt bt --heuristica=mrv

./horarios.sh instances/valid/small.dat \
    salida-fd.txt clpfd --heuristica=ff
```

Para buscar el menor costo se agrega `--optimizar`. El límite permite detener una búsqueda extensa.

```
./horarios.sh instances/valid/optimization.dat \
    salida-optima.txt clpfd --optimizar --limite=5000000
```

Para incluir las aulas y exportar el horario a Prolog se utilizan las opciones correspondientes.

```
./horarios.sh instances/valid/small.dat salida.txt bt \
    --horarios-por-aula --exportar-prolog=horario.pl
```

3. Archivo de entrada

La entrada es un archivo `.dat` en UTF-8 con saltos de línea de Linux. Los campos se separan con punto y coma. Las líneas vacías y los comentarios que empiezan con `#` se ignoran.

Las secciones pueden aparecer en cualquier orden, una sola vez cada una. Los valores numéricos son enteros no negativos y los identificadores usados deben estar declarados.

3.1. Configuración

La sección `CONFIG` contiene cuatro datos obligatorios. Este ejemplo define dos días, cuatro franjas diarias, una hora de inicio y franjas de sesenta minutos.

```
#:CONFIG
dias;L,M
franjas;4
inicio;07:00
duracion_franja;60
```

Cada día tiene un código de un carácter. El orden de los códigos define el orden de los días en el horario.

3.2. Aulas, profesores y cursos

Los registros de las demás secciones siguen estos formatos.

```
#:AULA
codigo;capacidad;tipo

#:PROFESOR
codigo;nombre;max_franjas;disponibilidad

#:CURSO
codigo;nombre;grupo;inscritos;semanales;duracion;requiere

#:IMPARTE
curso;grupo;profesor
```

Los códigos de aula y profesor son únicos. Cada grupo se identifica por el código de curso y el grupo. En `CURSO`, `semanales` indica cuántas sesiones se necesitan y `duracion` cuántas franjas consecutivas ocupa cada sesión.

Los tipos admitidos son `teoria`, `laboratorio` y `mixta`. Un grupo de teoría puede usar aulas de teoría o mixtas. Uno de laboratorio puede usar aulas de laboratorio o mixtas. El requerimiento `mixta` admite cualquier tipo de aula. La capacidad debe alcanzar para todos los inscritos.

La disponibilidad del profesor se escribe con rangos como `L1-L4,M2-M3`. Si el campo queda vacío, tiene disponibilidad total. Se comprueba que cada sesión comience dentro de esa disponibilidad y que la carga semanal no supere `max_franjas`.

Cada grupo necesita al menos un profesor en `IMPARTE`. Si tiene varios, todos participan en cada sesión.

3.3. Restricciones

La sección `RESTRICCION` admite estos registros. Un valor como `L3` indica el día y la franja.

```
aula_fija;curso;grupo;aula
prohibida;curso;grupo;diaFranja
excluyentes;curso;grupo;otroCurso;otroGrupo
prefiere;curso;grupo;diaFranja;peso
compacta;curso;grupo;peso
```

- `aula_fija` obliga al grupo a usar el aula indicada.
- `prohibida` impide que una sesión cubra esa franja.
- `excluyentes` impide que dos grupos tengan clases al mismo tiempo.
- `prefiere` suma el peso al costo si ninguna sesión empieza en el día y la franja indicados.
- `compacta` suma el peso por cada franja libre entre sesiones del grupo en un mismo día.

Las tres primeras restricciones son obligatorias. Las dos últimas son preferencias y pueden incumplirse a cambio de aumentar el costo.

4. Lectura del resultado

El archivo de salida muestra las sesiones asignadas, los horarios por grupo y profesor, el costo y las medidas de búsqueda. Los mensajes aparecen en inglés.

Sin optimización se entrega el primer horario válido. Con `--optimizar`, el resultado es óptimo cuando la búsqueda termina sin alcanzar el límite. Si se alcanza el límite, puede entregarse el mejor horario completo encontrado hasta ese momento.

Estado	Código	Significado
`SOLUTION FOUND`	0	Se encontró un horario válido.
`NO SOLUTION`	2	La instancia no tiene solución.
`LIMIT REACHED`	3	Se alcanzó el límite de búsqueda. No se garantiza el menor costo.
`INVALID INSTANCE`	1	La entrada tiene errores de formato o de datos.
Error en terminal	1	Falló la lectura o escritura, los argumentos son incorrectos o se detectó una carga imposible antes de buscar.

El reporte usa un diagnóstico cuando la búsqueda no encuentra un horario o la entrada es inválida. Los errores de archivos y las inconsistencias detectadas antes de buscar se informan en la terminal.

4.1. Archivo Prolog

La exportación contiene un hecho `asignacion/7` por sesión. Sus argumentos son curso, grupo, profesores, aula, día, inicio y duración. Los identificadores aparecen como listas de códigos.

El archivo del ejemplo anterior se carga con este comando.

```
gprolog --consult-file horario.pl
```

Dentro del intérprete se consultan las asignaciones.

```
asignacion(Course, Group, Teachers, Room, Day, Start, Duration).
```

5. Otras formas de ejecución

5.1. Uso interactivo

El programa se carga en GNU Prolog.

```
gprolog --consult-file horarios.pl
```

En el intérprete, `main/2` recibe los mismos argumentos en una lista y devuelve el código de salida.

```
main(['instances/valid/small.dat',
    'salida-interactiva.txt', bt], ExitCode).
```

Las opciones también pueden agregarse a esa lista. El intérprete permanece abierto y se cierra con `halt.`

5.2. Programa compilado

La versión compilada se prepara y ejecuta con estos comandos.

```
gplc --new-top-level -o horarios horarios.pl

./horarios --entry-goal main -- \
    instances/valid/small.dat salida-compilada.txt bt
```

Se usan las mismas opciones que con `horarios.sh`. Si cambia el código, se vuelve a compilar.

6. Pruebas y experimentos

Las instancias incluidas están en `instances`.

Archivo dentro de `instances`	Resultado esperado
`valid/small.dat`	Dos sesiones y costo cero.
`valid/constraints.dat`	Cinco sesiones y costo óptimo cero.
`valid/optimization.dat`	Costo óptimo 5.
`invalid/syntax.dat`	Error de formato.
`invalid/coherence.dat`	Errores en los datos.
`unsatisfiable/load.dat`	Carga imposible de grupo y profesor.
`unsatisfiable/rooms.dat`	Sin solución por falta de espacio.
`unsatisfiable/restrictions.dat`	Sin solución por grupos excluyentes.
`benchmark/large.dat`	Ochenta sesiones al encontrar una solución.

Los costos óptimos se comprueban con `--optimizar`. Para medir una ejecución también puede usarse GNU Time.

```
/usr/bin/time -p -o tiempos.txt \
    ./horarios.sh instances/valid/constraints.dat \
    resultado.txt bt --heuristica=mrv --limite=50000
```

Se repite la prueba con otras estrategias, heurísticas y con el programa compilado, manteniendo la misma instancia, el límite y la opción de optimización. El resultado incluye nodos, retrocesos y tiempos. `tiempos.txt` registra el tiempo del proceso completo.

Para comparar con backtracking básico se cambia el hecho de selección en `src/solvers/bt/search.pl`.

```
btSearchVariant(basic).
```

Esta variante usa el orden original y comprueba las restricciones duras al completar el horario. Al terminar la comparación se restaura `forward` y se recompila si se usa la versión compilada.

7. Si una ejecución falla

El mensaje de la terminal o el diagnóstico del archivo indican qué datos revisar. Alcanzar el límite no significa que la instancia no tenga solución.

Si una búsqueda grande agota la pila global, se puede aumentar su tamaño. Este ejemplo reserva 256 MiB.

```
GLOBALSZ=262144 ./horarios.sh instances/benchmark/large.dat \
    salida-grande.txt clpfd --heuristica=mrv --limite=50000
```

El límite de nodos no limita el tiempo ni la memoria de la ejecución.